Raytracer : Developers documentation

Here you will find the documentation to help you implement your own primitves and materials to Raytracer.

First, we will explain how to add a primitive to Raytracer, and then we will walk you through implementing your own materials.

I. How does Raytracer works

The goal of Raytracer is to simulate the effect of lights on objects, which are named “primitives”.

To make the project architecture clearer, we have provided an UML diagram inside the /doc folder, containing all of our interfaces and classes, helping you better understand how they interact with one another.

If you want to contribute to our project, you can create .cpp files in the src/ folder, and then add the filname to the CMakelist.txt.
If you want to add a new .hpp file, add it in the include/ folder.

Then, go to the build/ folder, and run :

```

cmake .. && make && ./raytracer > file.ppm && display file.ppm

```

II. How to implement a new primitve

Now that you know how Raytracer works, we will see how to implement new features. First, I will explain how to create a primitive compatible with our IPrimitve class.

A. hits()

The hardest part of a primitive : the hits method determines whether a ray intersects the primitive, and if so, at what distance. It returns a double, which represents the distance between the ray and the intersection point (or hit point).

If the ray (light ray) does not intersect with the primitive, you have to return -1.

Else, return the distance. For each primitive the formula is different, so you will have to figure out by yourself how to calculate this value, named “t”.

I strongly recommend you to follow this doc since it’s the best for raytracer :

https://raytracing.github.io/books/RayTracingInOneWeekend.html

B. getMaterial()

The getMaterial method returns an IMaterial. This method is used to apply the material and its properties (color, reflection, refraction, etc.) to your primitive.

C. getName()

This function must return the name of the primitive, in order to find it inside the IPrimitive factory's map. For example, if you want to create a plane, you must name it "plane".

D. getPosition()

Return the position of the primitive.

C. getColor()

Return the color of the primitive.

III. How to implement a new material

Perhaps you want to improve Raytrace by implementing a new material. The following section will show you how to do so. If you feel lost regarding the project's architecture, do not hesitate to look at the UML diagrams provided in the /doc directory.

A. calculateAmbient()

This method returns a factor of reflection for the ambient light. Since the goal of a material is to interact with lights, it must affect all the lights interacting with it.

B. calculateNewLight()

This method returns a factor of reflection for the direct light. If you cast a ray on the material, this method should return a percentage of reflection for this ray.

C. getRayNewColor()

Changes the color of the ray. Since the object reflects/refracts light, it must also change the inherent color of the light by a factor of its color.

IV. Conclusion

To conclude, this project is highly modular and allows for the addition of both Materials and Primitives. If you want to add yours, make sure to respect the IMaterial or IPrimitive interface classes, in order to use our factories.